DATA STRUCTURE LAB

# Array 與 ArrayList

// 固定格子，還是會長大的格子？

**length**

配置後固定

**size ≠ capacity**

看得見的元素 ≠ 底層空間

# 先看表面 API

```java
int[] scores = {85, 92, 78};
scores[1] = 95;
int n = scores.length;
```

索引語法 `[]` · 長度固定

```java
List<Integer> scores = new ArrayList<>();
scores.add(85);
scores.set(0, 95);
int n = scores.size();
```

方法 API · 元素數可變

# size 與 capacity 不是同一件事




**size = 3**

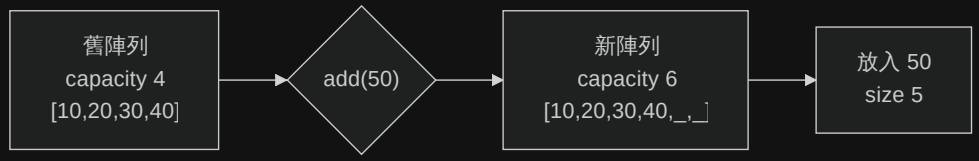
使用者可存取的元素數

**capacity = 4**

底層陣列目前的格數

capacity 是實作細節，標準 List API 不提供直接讀取。

# 容量滿了：建立新陣列再複製




**4 → 6**

4 + floor(4 / 2)

**複製 4 次**

這次 add 是 O(n)

**攤銷 O(1)**

不是每次都擴容

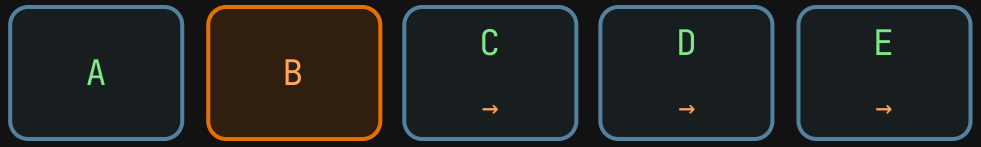
OpenJDK 常見成長約 1.5×；精確策略不是 List 規格保證。

# 看程式碼演進：預先給容量

```
1  List<Order> orders = new ArrayList<>(incoming);
```

已知資料量時，合理預估容量可減少重複配置與複製。

# 中間插入：成本來自「搬家」




**尾端 add**

通常不搬既有元素

**index 1 插入**

後方元素右移：O(n)

# 操作底層模型

Array：固定 length

ArrayList：size / capacity




size = 3 · capacity = 4 · 複製 0 · 搬移 0

40 add 1 insert 重設

拖曳元素可重新排序

先填滿 capacity 再 add；接著 insert、remove，最後拖曳元素觀察搬移計數。

# primitive 與 wrapper

```java
int[] raw = {1, 2, 3};
// 直接存 int 值
```

Array 可直接存 primitive：`int`、`double`、`char`

```java
List<Integer> boxed = new ArrayList<>();
boxed.add(1);          // int → Integer
int n = boxed.get(0);  // Integer → int
```

泛型不能使用 `List<int>`；需要 wrapper 與 autoboxing。

# 拆箱的隱藏風險

```java
List<Integer> values = new ArrayList<>();
values.add(null);

Integer boxed = values.get(0); // 合法：boxed == null
int raw = values.get(0);       // NullPointerException

Integer maybe = values.get(0);
if (maybe != null) {
    int safe = maybe;
    System.out.println(safe);
}
```

例外不是 get() 本身造成，而是 `Integer → int` 拆箱時發生。

# Arrays.asList 的固定大小陷阱

```java
List<String> readOnly = List.of("Ada", "Linus");
// set / add 都不允許
```

`Arrays.asList` 是陣列的固定大小 view，不是一般 ArrayList。

# 可編輯：選擇正確結構

```java
import java.util.*;

class Roster {
  public static void main(String[] args) {
    List<String> names =
        new ArrayList<>(List.of("Ada", "Linus"));
    names.add("Grace");
    System.out.println(names);
  }
}
```

### 練習

1. 改成固定長度 String[]
2. 比較加入第三人的寫法
3. 此 Monaco 僅編輯，不執行 Java

# 成本速查

| 操作 | Array | ArrayList |
| --- | --- | --- |
| `get(index)` | O(1) | O(1) |
| `set(index)` | O(1) | O(1) |
| 尾端 `add` | 不支援 | 攤銷 O(1) |
| 中間插入/刪除 | 手動重建 O(n) | O(n) |
| 擴容 | 不會 | 單次 O(n) |

Big-O 描述成長趨勢；boxing、快取與配置仍會影響實際常數。

# 選擇題：哪一個更自然？

**一週七天溫度**

固定 7 格、primitive 數值

`double[]`

**待辦清單**

持續新增與刪除

`ArrayList`

**棋盤 8 × 8**

尺寸固定、索引明確

`char[][]`

**搜尋結果**

數量事前未知

`ArrayList`

固定尺寸與 primitive 密集資料偏 Array；動態集合操作偏 ArrayList。

# 三句話帶走

> **size** 是元素數，**capacity** 是底層空間。

> 擴容約 1.5×，偶爾複製；中間操作需要搬移。

> primitive 與可變性，決定許多 API 陷阱。

# 謝謝

下一步：LinkedList、Set、Map 與效能量測

choose by behavior, measure by evidence